

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Monitoreo del estado de la batería con BroadcastReceiver en Jetpack Compose				
NÚMERO DE PRÁCTICA:	4	AÑO LECTIVO:	2026B	NRO. SEMESTRE:	VIII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	2	
FECHA INICIO:	22/09/2026	FECHA FIN:	26/09/2026	DURACIÓN:	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender qué es un BroadcastReceiver y para qué se utiliza en el desarrollo de aplicaciones Android. - Implementar un BroadcastReceiver que escuche un evento del sistema, como el cambio de estado de la batería. - Aplicar DisposableEffect para registrar y desregistrar el receiver de forma segura dentro del ciclo de vida de un Composable.	
TEMAS: - BroadcastReceiver y eventos del sistema - DisposableEffect: efectos con limpieza en Compose - PendingIntent para el envío de broadcasts personalizados	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Video: "Broadcast Receivers en Android con Jetpack Compose"

<https://www.youtube.com/watch?v=AeNnc8yhNT0>

- Android Developers, monitoreo del estado de la batería:

<https://developer.android.com/training/monitoring-device-state/battery-monitoring>

¿Qué es un BroadcastReceiver?

Un BroadcastReceiver es un componente de Android que permite a una aplicación recibir mensajes (Intents) que el sistema operativo, u otras aplicaciones, envían de forma general (broadcast). Se utiliza, por ejemplo, para enterarse de que cambió el estado de la batería, que se conectó o desconectó la red, o que terminó una descarga. A diferencia de una pantalla, un BroadcastReceiver no tiene interfaz propia: simplemente reacciona a un evento y puede actualizar el estado de la aplicación.

Registro dinámico del receiver

Un BroadcastReceiver puede registrarse de forma dinámica en tiempo de ejecución con `context.registerReceiver(receiver, IntentFilter(...))`, y debe desregistrarse con `context.unregisterReceiver(receiver)` cuando ya no se necesita. Si no se desregistra correctamente, la aplicación puede seguir escuchando eventos incluso cuando esa parte de la interfaz ya no está visible, lo que produce fugas de memoria.

DisposableEffect: efectos con limpieza en Compose

Un Composable no tiene métodos como `onResume()` u `onPause()`; en su lugar, Compose ofrece `DisposableEffect` para ejecutar código cuando el Composable entra en la pantalla, y código de limpieza (dentro de `onDispose { }`) cuando sale de ella. Es el mecanismo recomendado para registrar y desregistrar un BroadcastReceiver de forma segura, como se resume en la Figura 1.

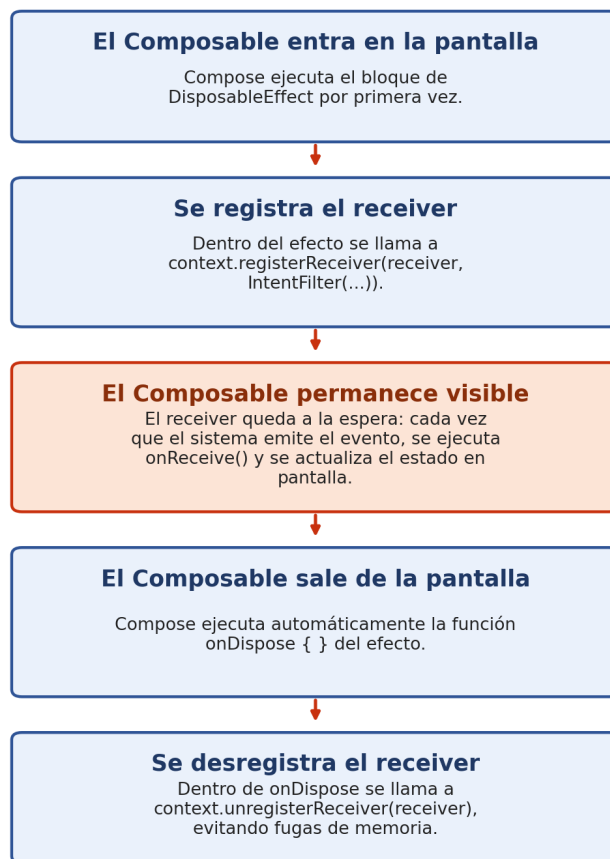


Figura 1. Ciclo de registro y desregistro de un receiver con DisposableEffect (elaboración propia).

Monitoreo del estado de la batería

El sistema Android emite el Intent implícito ACTION_BATTERY_CHANGED cada vez que cambia el estado de carga del dispositivo. Este Intent incluye, entre otros datos, EXTRA_LEVEL (nivel actual) y EXTRA_SCALE (nivel máximo posible), con los que se puede calcular el porcentaje de batería. La Figura 2 resume el flujo completo, desde que el sistema emite el evento hasta que se actualiza la interfaz.

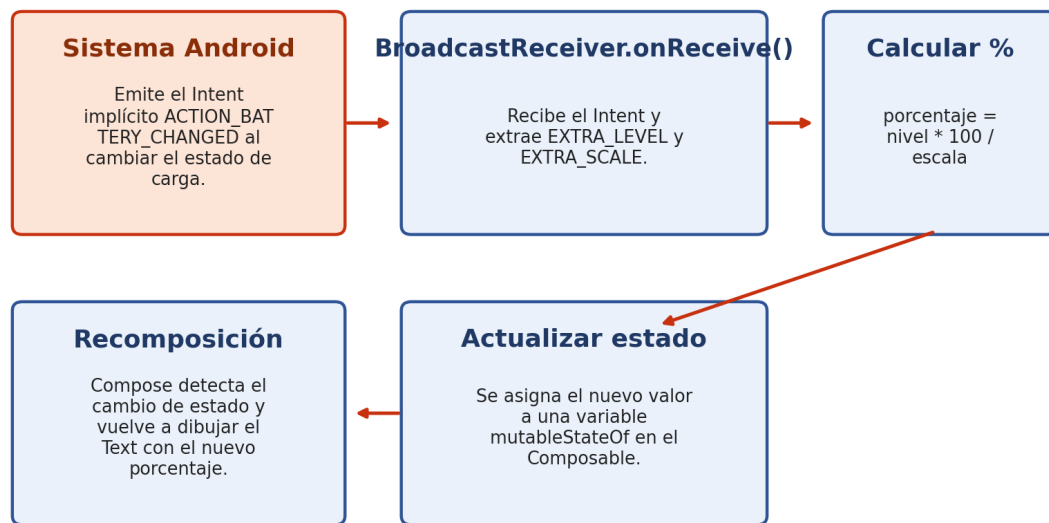


Figura 2. Flujo del BroadcastReceiver de batería, del evento del sistema a la actualización en pantalla (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para mostrar el porcentaje de batería en pantalla usando un BroadcastReceiver registrado con DisposableEffect. También puede apoyarse en el siguiente video tutorial:

Video de referencia: "Broadcast Receivers en Android con Jetpack Compose"

<https://www.youtube.com/watch?v=AeNnc8yhNT0>

Paso 1. Crear un nuevo proyecto con la plantilla "Empty Activity" y crear una función @Composable llamada BatteryScreen().

nombre sugerido para el proyecto: "BatteryMonitor_Compose", Package name: com.example.batterymonitor_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Código:

```
@Composable
fun BatteryScreen() {
    // Contenido en los siguientes pasos
}
```

Importaciones que se usarán en los siguientes pasos (agregar junto a las demás importaciones del archivo):

```
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.os.BatteryManager
import android.util.Log
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 5

```
import androidx.compose.runtime.setValue
import androidx.compose.ui.platform.LocalContext
```

Paso 2. Dentro de BatteryScreen, declarar una variable de estado: var porcentaje by remember { mutableStateOf(0) }.

Código (dentro de BatteryScreen):

```
var porcentaje by remember { mutableStateOf(0) }
```

Paso 3. Obtener el contexto actual con val context = LocalContext.current, necesario para registrar el receiver.

Código (dentro de BatteryScreen, después de la variable de estado):

```
val context = LocalContext.current
```

Paso 4. Dentro de un DisposableEffect(Unit), crear un objeto que extienda BroadcastReceiver. En su método onReceive(), extraer EXTRA_LEVEL y EXTRA_SCALE del Intent recibido, calcular el porcentaje (nivel * 100 / escala) y actualizar la variable de estado.

Código:

```
DisposableEffect(Unit) {
    val receiver = object : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val nivel = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1
            val escala = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1
            if (nivel != -1 && escala != -1) {
                porcentaje = (nivel * 100) / escala
            }
        }
    }

    // El registro del receiver se agrega en el Paso 5
}
```

Paso 5. Dentro del mismo DisposableEffect, registrar el receiver con context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED)), e imprimir por consola un mensaje indicando que se registró correctamente.

Código (agregar dentro del mismo DisposableEffect, justo después de crear "receiver"):

```
context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
Log.d("BatteryScreen", "Receiver registrado")
```

Paso 6. Cerrar el DisposableEffect con onDispose { context.unregisterReceiver(receiver) }, imprimiendo también un mensaje de confirmación por consola, como se resume en la Figura 1.

Código (cierra el DisposableEffect):

```
onDispose {
    context.unregisterReceiver(receiver)
    Log.d("BatteryScreen", "Receiver desregistrado")
}
```

Código completo del DisposableEffect (Pasos 4 a 6 juntos):

```
DisposableEffect(Unit) {
    val receiver = object : BroadcastReceiver() {
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 6

```

        override fun onReceive(context: Context?, intent: Intent?) {
            val nivel = intent?.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) ?: -1
            val escala = intent?.getIntExtra(BatteryManager.EXTRA_SCALE, -1) ?: -1
            if (nivel != -1 && escala != -1) {
                porcentaje = (nivel * 100) / escala
            }
        }
    }

    context.registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
    Log.d("BatteryScreen", "Receiver registrado")

    onDispose {
        context.unregisterReceiver(receiver)
        Log.d("BatteryScreen", "Receiver desregistrado")
    }
}

```

Paso 7. En el cuerpo de BatteryScreen, mostrar el valor con un Text("Batería: \$porcentaje").

Código (en el cuerpo de BatteryScreen, fuera del DisposableEffect):

```
Text("Batería: $porcentaje")
```

Paso 8. En MainActivity, dentro de setContent { }, reemplazar la llamada a Greeting(...) que trae la plantilla por defecto por BatteryScreen(), para que esta pantalla sea la que se muestra al ejecutar la app.

Código para MainActivity.kt:

```

setContent {
    BatteryMonitor_ComposeTheme {
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
            BatteryScreen()
        }
    }
}

```

Paso 9. Ejecutar la aplicación en un emulador y, desde Extended Controls > Battery, simular distintos niveles de carga, verificando que el porcentaje mostrado en pantalla se actualiza siguiendo el flujo de la Figura 2.

Este paso no requiere código adicional; ejecuta la app (Run) y, desde Extended Controls > Battery del emulador, simula distintos niveles de carga para verificar que el porcentaje en pantalla se actualiza.

Paso 10. Revisar Logcat y confirmar que aparecen los mensajes de registro y desregistro del receiver al entrar y salir de la pantalla.

Este paso no requiere código adicional; basta con abrir Logcat, filtrar por la etiqueta "BatteryScreen" y confirmar que aparecen los mensajes de registro y desregistro al entrar y salir de la pantalla.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Tomando como referencia el ejercicio resuelto por el docente, agrega una segunda forma de actualizar el porcentaje de batería, disparada manualmente en lugar de por el evento automático del sistema:

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

1. Agrega un botón "Actualizar manualmente" a BatteryScreen. Al presionarlo, en vez de esperar el evento del sistema, dispara un broadcast personalizado: crea un Intent con una acción propia (por ejemplo "com.tuapp.ACTUALIZAR_BATERIA"), envuélvelo en un PendingIntent.getBroadcast(...), y actívalo con PendingIntent.send().
2. Implementa un segundo BroadcastReceiver (o reutiliza el mismo patrón) que escuche esa acción personalizada y, al recibirla, vuelva a leer el estado de la batería (por ejemplo con BatteryManager) y actualice el mismo estado en pantalla.
3. Registra y desregistra este segundo receiver siguiendo el mismo patrón de DisposableEffect usado en el ejercicio resuelto.
4. (Reto opcional) Muestra en pantalla, además del porcentaje, si la batería se está cargando o no, usando EXTRA_STATUS del Intent original.

Entregable: informe con capturas de pantalla mostrando el porcentaje actualizándose automáticamente (evento del sistema) y manualmente (botón + PendingIntent), y una breve descripción de la implementación.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantalla que muestra el porcentaje de batería en tiempo real)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. En tu propia implementación, ¿qué diferencia encontraste entre recibir el evento automático del sistema (ACTION_BATTERY_CHANGED) y disparar un broadcast manualmente con PendingIntent? ¿En qué situaciones usarías cada uno?
3. En forma individual, comenta en el video de referencia (<https://www.youtube.com/watch?v=AeNnc8yhNTo>) qué otras acciones del sistema, además del cambio de batería, se pueden escuchar con un BroadcastReceiver. (Colocar la captura del comentario; uno por integrante del grupo).

V. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- [1] Android Developers, "Descripción general de broadcasts". [En línea]. Disponible: <https://developer.android.com/develop/background-work/background-tasks/broadcasts>
- [2] Android Developers, "Battery monitoring". [En línea]. Disponible: <https://developer.android.com/training/monitoring-device-state/battery-monitoring>
- [3] Android Developers, "Side-effects in Compose". [En línea]. Disponible: <https://developer.android.com/develop/ui/compose/side-effects>
- [4] "Broadcast Receivers en Android con Jetpack Compose", video. [En línea]. Disponible: <https://www.youtube.com/watch?v=AeNnc8yhNTo>

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS: Ejercicios propuestos	INSTRUMENTOS: Rúbrica de evaluación
---	---

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 8

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.